Quarter-Decay ratio? P, PI, PID controllers?

10/02/24

Artificial Neural Network (ANN):-

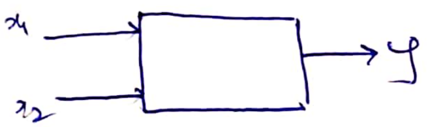
LMSE $\rightarrow \frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2$

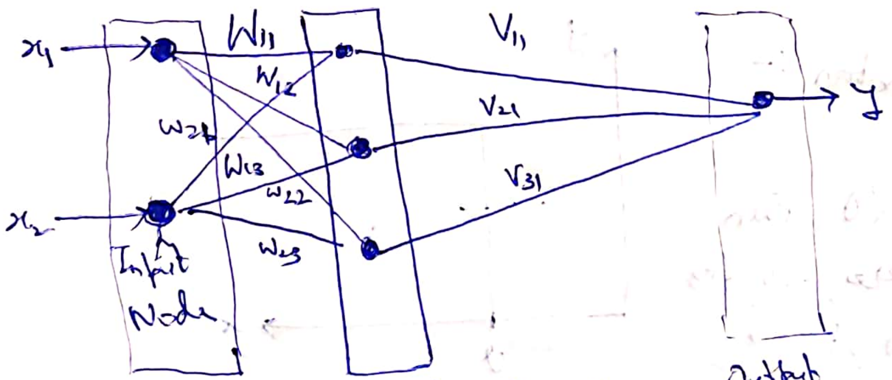
least mean square error, curve → linear regression. so trying to fit a+bx to a data. the normal equations are:-

$$\sum y_i = na + b\sum x_i$$

$$\sum x_i y_i = a\sum x_i + b\sum x_i^2$$

11 → 1st input node to 1st hidden node.





Input layer
l
↓
no. of nodes
(No. of nodes)

Hidden layer/s
m (m)
↓
no. of hidden nodes
(No. of hidden nodes)

Output layer
n

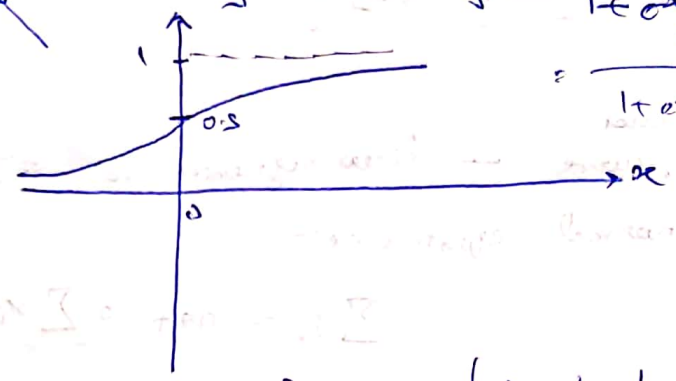
Number of connections (arcs) = $l \times m$, so, [W] is a matrix of $(l \times m)$. Similarly, [V] will be a matrix of $(m \times n)$.

Sigmoidal fn.
(Transfer function)

Sigmoidal function is also called the Sigmoidal function

transfer function in the node.

$$y = \frac{1}{1+e^{-\alpha x}}$$



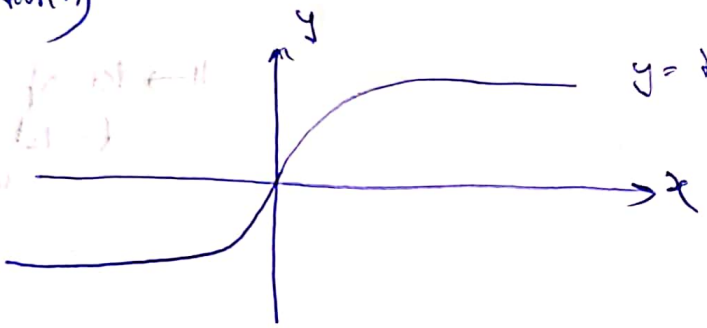
$$y = \frac{1}{1+e^{-\alpha x}}$$

$\alpha \rightarrow$ parameter

α is a parameter

Also, Hyperbolic tangent is also used as a transfer function

(tanh)



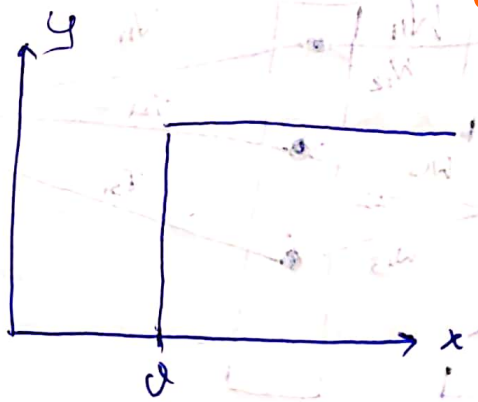
$$y = \tanh x = \frac{e^{mx} - e^{-mx}}{e^{mx} + e^{-mx}}$$

$$\tanh x = \frac{e^{mx} - e^{-mx}}{e^{mx} + e^{-mx}}$$

like $\sinh x / \cosh x$

Threshold function :-

Till input < 0, then output y is zero, only if $x > 0$, then we do get the response



Input to 1st node of hidden layer $\rightarrow I_1 = \sum W_{i1} O_i$

- here $O_{i1} \rightarrow x_i$ as first layer the output is same as input in the input layer or may be not.

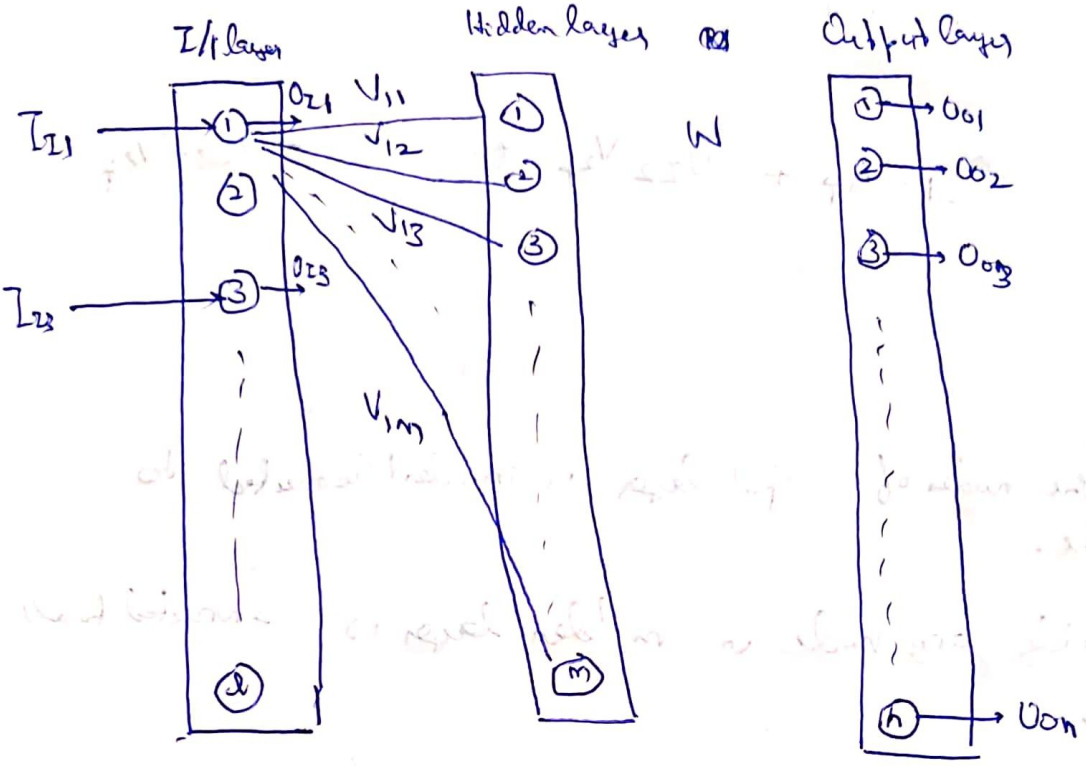
$I_{I,0}$ → $O_{I,1}$

$O_{H,3}$

$I_{H,3}$

$O_{O,K}$

$I_{O,K}$



If input is incident to input layer node then I_I,
input is incident to hidden layer node then I_H,
input is incident to output layer node then I_O.

I_{Ii}, I_{Hj}, I_{Ok} i, j, k are the index of the node in that layer

The output follows the same notation convention, O_I, O_H, O_O.

$O_{I1} = f(I_{I1})$ → could be a sigmoid.

The first node of input layer is connected to all the hidden layer nodes.

Input layer Computation :-

$\{O\}_I = \{I\}_I$ (transfer function is 1)

↑ output vector of input layer

output vector of input layers → $\begin{bmatrix} O_{I1} \\ O_{I2} \\ \vdots \\ O_{Il} \end{bmatrix}$

$$I_{Hp} = O_{I1}\cdot V_{1p} + O_{I2} V_{2p} + \dots + O_{Il}\cdot V_{lp}$$

(p is any arbitrary node in hidden layer)

p → arbitrary node in hidden layer

Also all the nodes of input layer is ~~incident~~/connected to pth node.

So, basically any node in hidden layer is connected to all the nodes.

p → 1 to m, so we could write m such equations for I_{H}s,

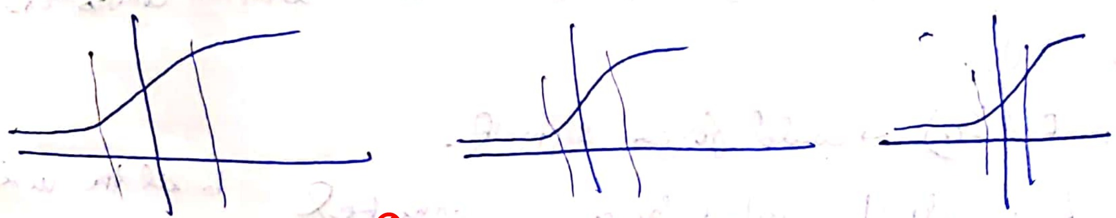
→ matrix form of eqns

$$\{I\}_H = [V]^T_{(m\times l)} \{O\}_{I(l\times 1)}$$

$V_{l\times m} \to V^T_{m\times l}$

$$O_{Hp} = \frac{1}{1+e^{-\lambda I_{Hp}}}$$

, λ the sharpness of the slope defines the band where the curve is sensitive



$$O_{Hp} = \frac{1}{1+e^{-\lambda I_{Hp}}}$$

$$\{O\}_H = \{f(I)\}_H$$

Again, O_{HP} will go to all the nodes in the each output layer nodes with associated weights $W_{p1}, W_{p2}, \ldots W_{pn}$

$$I_{oq} = O_{H1} W_{1q} + O_{H2} W_{2q} + \cdots + O_{Hm} W_{mq},$$

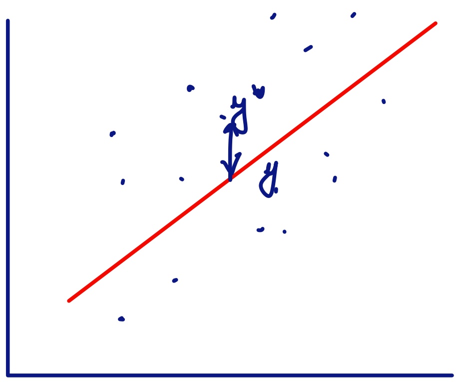
$q = 1, 2, 3 \ldots n$

$$\{I\}_o = [W]^T_{n \times m} \{O\}_{H\,(m \times 1)}$$

$$O_{oq} = \frac{1}{1 + e^{-\lambda I_{oq}}}$$

$$\{O\}_o = \left\{ \frac{1}{1 + e^{-I_{oq}}} \right\}$$

Randomise data (If we seggregate based on sequence, prediction must be upto mark)
↳ Then do train test split



$\Sigma(y^* - y)^2$ → error need to minimise

$\Sigma(y - a - bx)^2 = S$

$\frac{\partial S}{\partial a} = 0 \quad \frac{\partial S}{\partial b} = 0$

I/P	O/P
X	Y
-	-
-	-
-	-
-	-

W, V → weights → of neural nets

$E = f(W, V)$ → error fn. of weights

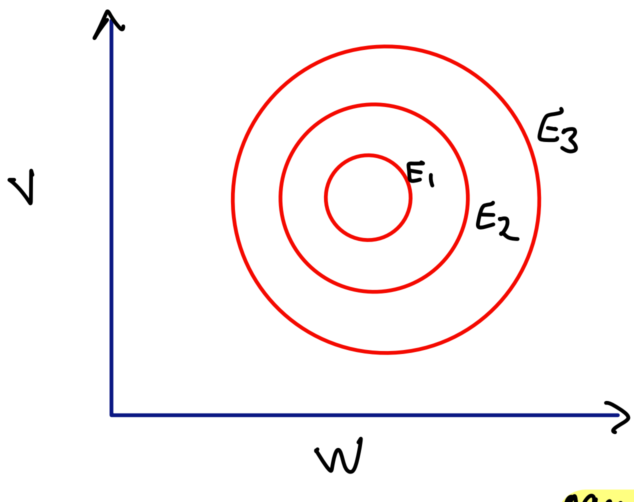
$\frac{\partial E}{\partial w} = 0 \quad \frac{\partial E}{\partial v} = 0$ E = RMSE

2/3 → Training → N
1/3 → Testing

X	Y	O_o → predicted
-	-	-
-	-	-
-	-	-

Y ↳ Actual

$\frac{\Sigma(Y - O)^2}{N}$ → need to minimised ↗ LMSE



$E_3 > E_2 > E_1$ → contours of error

Suppose we get E_3, when we chose W, V as random, then after iterations we got E_2 and finally E_1 so we say we have converged where E_1 is the tolerance and V, W can be employed in real time

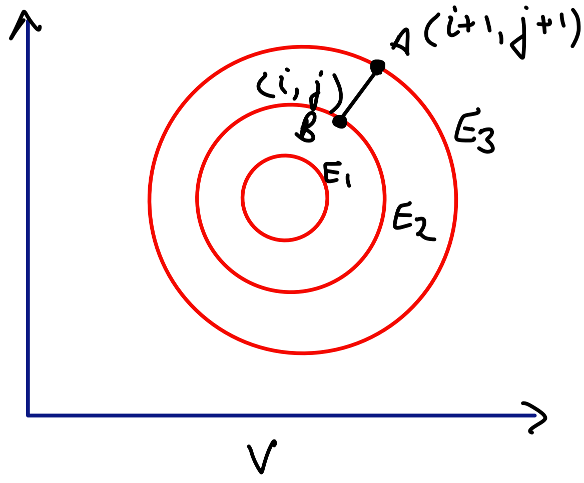
gradient descent method → at every point you have to find the steepest gradient

Hindi sabse jyada jis point pe error steeply reduce ho raha h

$$\vec{G} = \left(\frac{\partial E}{\partial v}\right)\hat{i} + \left(\frac{\partial E}{\partial w}\right)\hat{j}$$

$$\vec{AB} = -\eta\,[\vec{G}]$$

η → magnitude



$$\vec{AB} = (v_{i+1} - v_i)\hat{i} + (w_{j+1} - w_j)\hat{j}$$

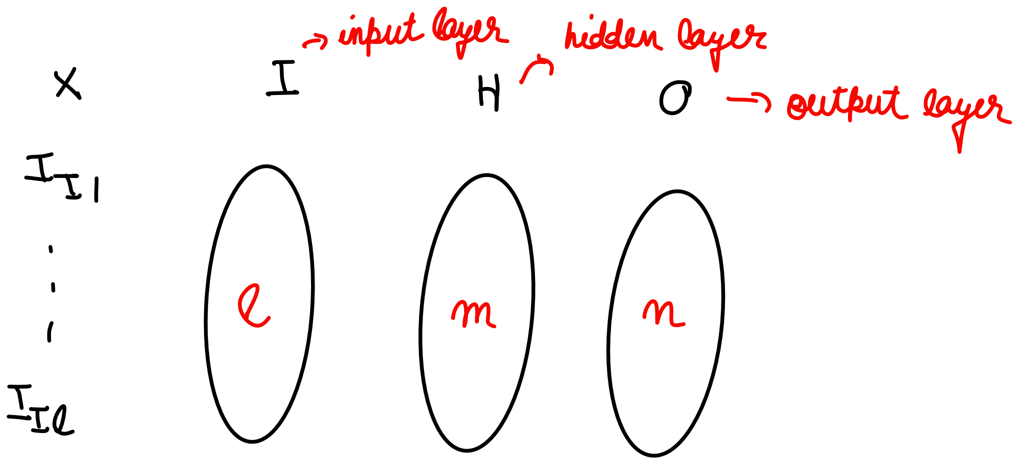
$$= (\Delta v)\hat{i} + (\Delta w)\hat{j}$$

$$\Delta v = -\eta\left(\frac{\partial E}{\partial v}\right)$$

$$\Delta w = -\eta\left(\frac{\partial E}{\partial w}\right)$$

→ Most important

$$E_k = \frac{1}{2}\left[T_k - O_{ok}\right]^2$$



	X	Y
1	I_{T1} . . . . . I_{Tl}	T_1 . . . . . T_n
⋮		
R		

$$E_k = \frac{1}{2}[T_k - O_{ok}]^2$$

basically we are trying to minimise errors by optimising weights of the layers.

T_k → actual, O_{ok} → predicted

E_k → error of k^{th} node

$$\frac{\partial E_k}{\partial w_{ik}} = \left(\frac{\partial E_k}{\partial O_{ok}}\right)\left(\frac{\partial O_{ok}}{\partial I_{ok}}\right)\left(\frac{\partial I_{ok}}{\partial w_{ik}}\right)$$

→ why chain rule?

↓

No direct relationship b/w E & W, and we have relationship b/w E, O, O, I & I, W

w_{ik} → Weight of i^{th} layer, k^{th} node

$$-(T_k - O_{ok})$$

$$O_{ok} = \frac{1}{1 + e^{-\lambda(I_{ok})}}$$

→ sigmoidal transfer fn.

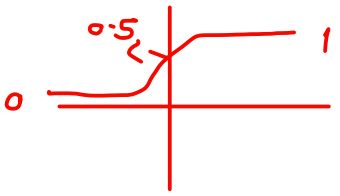
↓

can be used to make training more effective

↳ $y = \frac{1}{1+e^{-\lambda x}}$

$y \to [0, 1]$

pass through y axis at 0.5



→ Slope depends on λ

$$\frac{\partial O_{ok}}{\partial I_{ok}} = (\lambda O_{ok}(1 - O_{ok}))$$

can be changed by λ

sensitivity → changing x, y changes

insensitivity → " " , y does not change

$$I_{ok} = w_{1k} \cdot O_{H1} + w_{2k} \cdot O_{H2} + w_{ik} \cdot O_{Hi} \cdots + w_{mk} \cdot O_{Hm}$$

$$\frac{\partial I_{ok}}{\partial w_{ik}} = O_{Hi}$$

$$\frac{\partial E_k}{\partial w_{ik}} = -\lambda (T_k - O_{ok})\left[O_{ok}(1 - O_{ok})\right](O_{Hi})$$

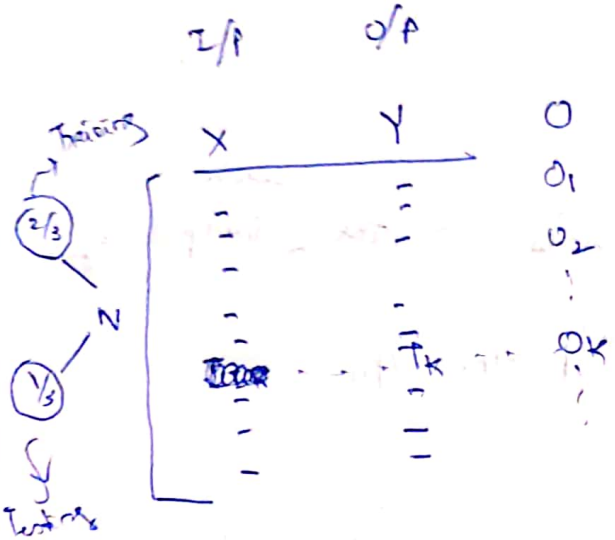
λ → tuning parameter

→ final expression

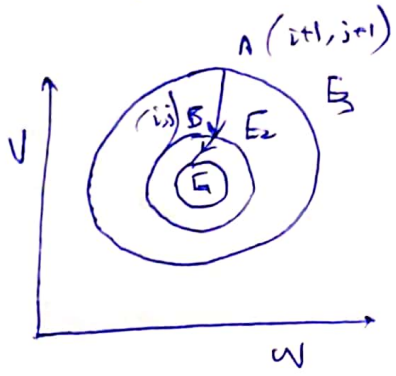
η → learning rate → if large then will diverge

η, λ → tuning parameters

4/03/24



V, W → weights in NN



$$\vec{G} = \left(\frac{\partial E}{\partial V}\right)\hat{i} + \left(\frac{\partial E}{\partial W}\right)\hat{j}$$

↓
gradient

$$\vec{AB} = -\eta[\vec{G}]$$

↓ magnitude of distance

$$\vec{AB} = (V_{i+1} - V_i)\hat{i} + (W_{j+1} - W_j)\hat{j}$$

$$= (V_{i+1} - V_i)\hat{i} + (W_{j+1} - W_j)\hat{j} = \Delta V\hat{i} + \Delta W\hat{j}$$

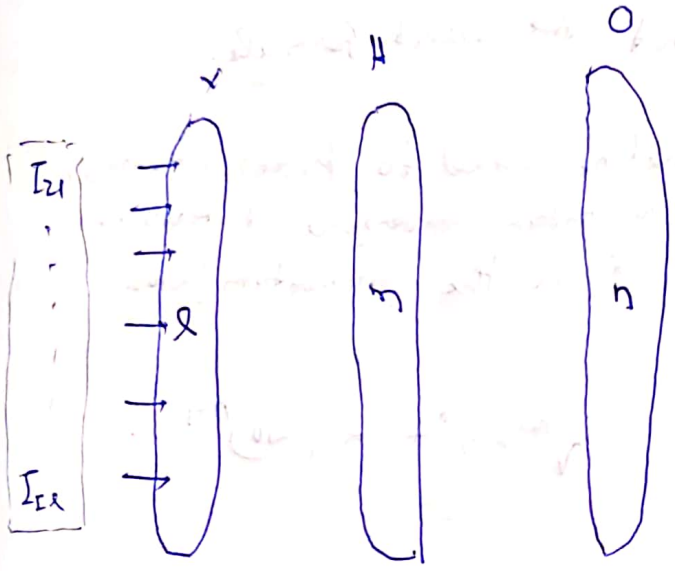
$$\Delta V = -\eta \left(\frac{\partial E}{\partial V}\right)$$

$$\Delta W = -\eta \left(\frac{\partial E}{\partial W}\right)$$

$E_K = \frac{1}{2}\left[T_K - O_{OK}\right]^2$

O_{OK} → Output of the kth node in the output layer

↓ for one data set of $\{I_{I1} \ldots I_{Ie}\}$



	X	Y
Ist input →	$I_{I1}, \ldots, I_{Ie}$	$T_1, \ldots T_n$
↓		
next		

$$\left(\frac{\partial E_K}{\partial W_{jK}}\right) = \left(\frac{\partial E_K}{\partial O_{OK}}\right)\left(\frac{\partial O_{OK}}{\partial W_{jK}}\right)$$

$$\left(\frac{\partial E_K}{\partial W_{iK}}\right) = \left(\frac{\partial E_K}{\partial O_{OK}}\right)\left(\frac{\partial O_{OK}}{\partial I_{OK}}\right)\left(\frac{\partial I_{OK}}{\partial W_{iK}}\right)$$

$= \partial\left[-\left(T_K - O_{OK}\right)\right]\left(\lambda O_{OK}(1 - O_{OK})\right) \times O_{Hi}\}$

← All the values here are known from forward calculation.

$$O_{OK} = \frac{1}{1 + e^{-\lambda I_{OK}}}, \quad I_{OK} = \sum_{j=1}^{m} H_{Oj} W_{jK}$$

or

$$= \sum_{j=1}^{m} O_{Hj} W_{jK}$$

5/03/24

$$\frac{\partial E_k}{\partial w_{ik}} = -\left[\lambda (T_k - O_{ok}) O_{ok} (1 - O_{ok})\right] \cdot O_{Hi}$$

(error representation; $d_k \rightarrow k^{th}$ term of $\langle d \rangle$ vector; λ → learning parameter)

Δw_{ik}

$$\Delta w_{ik} = -\eta \left(\frac{\partial E_k}{\partial w_{ik}}\right)$$

$$= \eta\lambda (T_k - O_k) O_{ok} (1 - O_{ok}) O_{Hi}$$

$$\Delta w = -\eta \frac{\partial E}{\partial w}$$

$\frac{\partial E}{\partial w}$ → calculated

↳ weight increment

$$W_i^{n+1} = W^n + (\Delta w)^{n+1} * \alpha$$

→ not the correct formula.

↳ sometime added to prevent overlearning to obtain convergence at iteration.

α is the momentum factor

$$[\Delta w]_{m \times n} = \eta \{O\}_{m \times 1} \cdot \langle d \rangle_{1 \times n}$$

learning rate; momentum factor

$$V^{n+1} = V^n + \alpha (\Delta v)^{n+1}$$

$$[\Delta v]_{l \times m} = \eta \{I\}_{l \times 1} \langle d^x \rangle_{(1 \times m)}$$

η → learning rate

α → momentum factor

$$d_i^x = \lambda e_i (O_{Hi}) (1 - O_{Hi})$$

$$\{ \cdot \}$$

$$[w_{ik} d_k]$$

Recurrent net, Reinforced learning

XOR Gate

X	Y	Z
0	0	0
1	0	1
0	1	1
1	1	0

(Neuro-fuzzy system)

Neural network helps in defining the membership function of the fuzzy logic.

neural network replaces membership value/function.

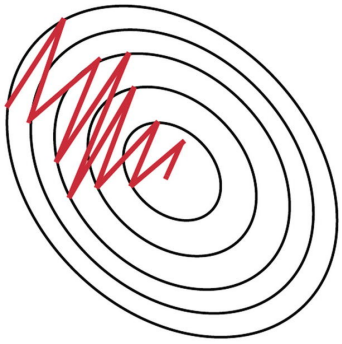
$$\Delta w_{ij} = (\eta * \frac{\partial E}{\partial w_{ij}})$$

weight increment — learning rate — weight gradient

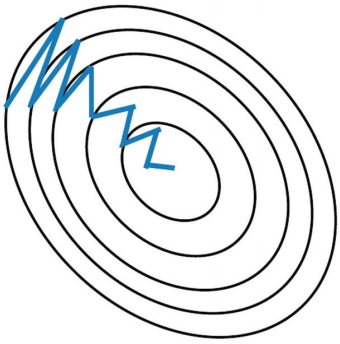
Important diagrams

$$\Delta w_{ij} = (\eta * \frac{\partial E}{\partial w_{ij}}) + (\gamma * \Delta w_{ij}^{t-1})$$

momentum factor — weight increment, previous iteration



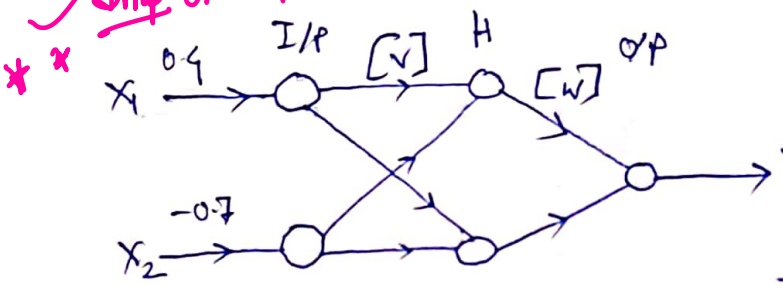
Stochastic Gradient Descent **without** Momentum



Stochastic Gradient Descent **with** Momentum

22/03/24

Imp Example **



O_0 = 0.4642 → from calculation.

To, from training

X_1	X_2	Y
0.4	-0.7	0.1
0.3	-0.5	0.05
0.6	0.1	0.3
0.1	-0.2	0.12

$$[V]^0 = \begin{bmatrix} 0.1 & 0.4 \\ -0.2 & 0.2 \end{bmatrix}_{2\times 2}$$

$V_{11} = 0.1$, $V_{12} = 0.4$, V_{21}, V_{22}

↓ initialization of the weights done arbitrarily

After obtaining new data we do randomization and normalization

$$[W]^0 = \begin{bmatrix} 0.2 \\ -0.5 \end{bmatrix}_{2\times 1}$$

W_{11}, W_{21}

data is normalized between -1 to +1 and hence weights are also -1 to +1.

i) $\{O\}_I = \{I\}_I$ → no transfer function in input layer

$$= \begin{Bmatrix} 0.4 \\ -0.7 \end{Bmatrix}$$

ii) Initialization of W, V → done

iii) Find input to the hidden layer

$$\{I\}_H = [V]^T \{O\}_I = \begin{bmatrix} 0.1 & -0.2 \\ 0.4 & 0.2 \end{bmatrix} \begin{Bmatrix} 0.4 \\ -0.7 \end{Bmatrix}$$

$$= \begin{Bmatrix} 0.18 \\ 0.02 \end{Bmatrix}$$

iv) $\{O\}_H = \begin{Bmatrix} \frac{1}{1+e^{-0.19}} \\ \frac{1}{1+e^{-0.02}} \end{Bmatrix} = \begin{Bmatrix} 0.545 \\ 0.505 \end{Bmatrix}$

v) $\{I\}_o = [W]^T \cdot \{O\}_H = [0.2 \quad -0.5] \cdot \begin{Bmatrix} 0.545 \\ 0.505 \end{Bmatrix}$

$= -0.1435$

vi) $\{O\}_o = \left[\frac{1}{1+e^{+0.1435}} \right] = 0.4642$

vii) Error $= (T_o - O_o)^2 = (0.1 - 0.4642)^2 = 0.13264$

viii) $\delta = (T_o - O_o)\, O_o\, (1 - O_o) = -0.0906$

$\frac{\partial E}{\partial W}$

$[Y] = \{O\}_H \cdot \langle \delta \rangle = \begin{Bmatrix} -0.0499 \\ -0.0457 \end{Bmatrix}$

Learning rate is associated with gradient descent

ix) $[\Delta W]^1 = \alpha [\Delta W]^0 + \eta [Y]$ → for 0th step there is not any but for $i = 1, 2, \ldots$, that value will be available.

α: momentum factor; $[\Delta W]^0$: change of W from previous iteration.

$\eta = 0.1$, $\alpha = 0.6$ → taking

$$[\Delta W]^1 = \begin{bmatrix} -0.0296 \\ -0.0274 \end{bmatrix}$$

$$\langle X \rangle \quad [W]^1 = [W]^0 + [\Delta W]^1$$

$$= \begin{bmatrix} 0.1704 \\ -0.5274 \end{bmatrix}$$

Basically it's ANN based mould breakout prediction system (For continous casting of steel)

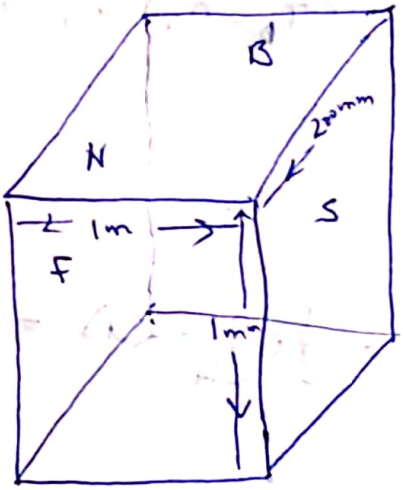
mold, primary zone

mold/primary zone → breakout → breakout detection system

ANN based MBDS

F, B → (1 × 1) m

(N, S) → (1 × 200 mm)

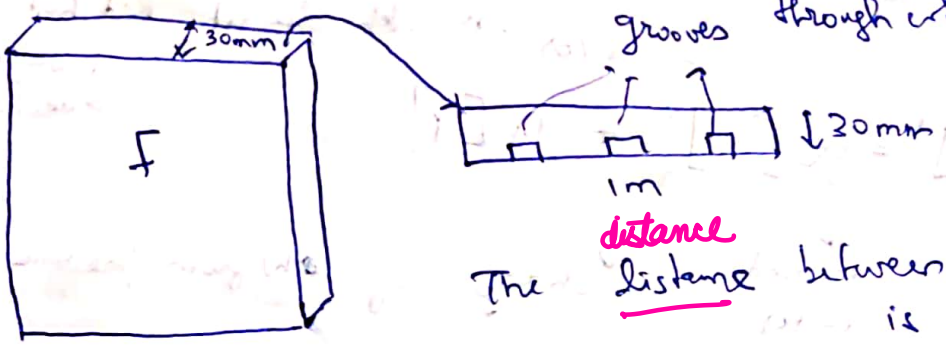


This is the mold

→ This is the mold

there are 4 plates F, B, N, S

So, taking the front plate :-

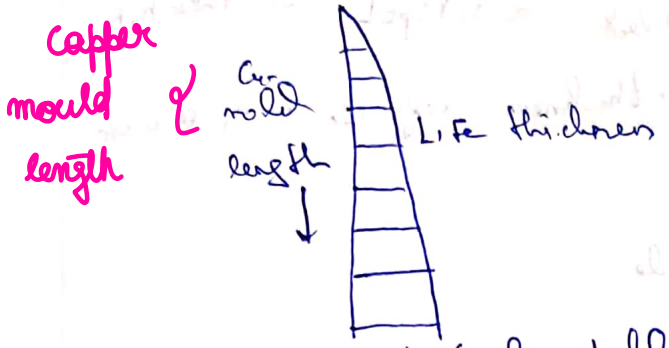


which water circulates through which water circulates

The distance between F and B plate is 200 mm.

This water circulation leads to indirect cooling of almost 2.5 MW magnitude

As the liquid Fe goes through the mould, a solidification/shell formation takes place.



after leaving the mold. When it comes out of the mould, the shell thickness to be sufficient to withstand the ferrostatic head as there are no walls in the mold to support it anymore

This possible that one of the grooves could be clogged, so that portion of the liquid steel will not be solidified as expected. The thickness will be less. So, this mould breakout has to be detected.

One we detect the small thickness, identify the weak point, our control action is to decrease the casting speed, so we are allowing more residence time inside the mould. This reduced speed is called creep speed. → creep speed

If we can detect a breakout between, the top 1/3rd part of the mould, then will have enough time to implement the control action and then ensure that our creep speed is reached the holding time is sufficient that the steel coming out has the desired structural integrity.

If, the thickness is low, then liq steel will be closer to copper walls, and therefore temperature of the flesh on ~~that~~ ~~heat~~ region will be higher, often called a hotspot. So, these hotspots are used to identify low shell-thickness regions, using sensors and thermocouples.

1 Level – 32 Thermocouple
3 levels

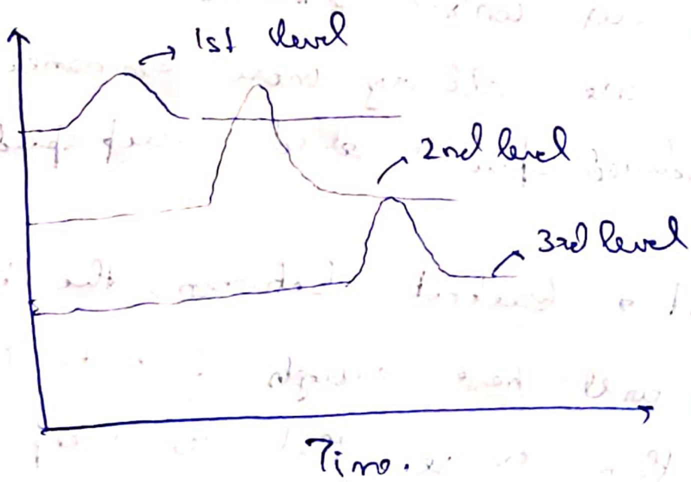
Fand B → 2×12 thermocouples (each for each side)
N and S → 2× 4 T.C.

} for 1 level and there are 3 levels.
32 T-C

These levels are within the 1/3rd region

The shell offer resistance so more shell thickness less temp.

As soon as temp rise / hotspot (sticker) is detected in one of the thermocouples. But the hotspot is also moving so, after sometime it will reach the 2nd level.



The amplitude gives us an idea whether it is working or not. If the amplitude is decreasing then the healing is taking place.

So, a ANN will be trained for pattern recognition, and seeing these graphs they will be able to tell if there will be a break-out / not.

It is possible that there is an actual hotspot that the ANN would not detect.

Also, there is no hotspot but ANN says there is.

13/03/24

Genetic Algorithm :-

maximise f(x) → this is an optimization problem.

standard design/procedure:- $\frac{\partial f}{\partial x} = 0$ → gradient based method.

subj to ... constraints, so constrained optimization.

This comes from darwin's theory - survival of the fittest.

New species from old species will comes from:-

i) Reproduction

ii) Cross-Over

iii) Mutation

Steps:-

i) Population - each member is a potential optimal point.

ii) Enrich population:- (reproduction)

1 ⇉ replicate the toppers to obtain 6 members → 1, 2, 3, 4

2

3

4

---

5 ↘ failure

6 ↗

3) Undergoes fitness test as before
↓
assign a fitness function, reject as well

4) undergoes mutation and cross over.
↓ ↓
How many members will undergo
is a tunable parameter

5) We will obtain a new population. range between which we will get the optimum values

The numbers lets say is between Cmin and Cmax

$$C_i = C_{min} + \frac{b}{(2^L - 1)} (C_{max} - C_{min})$$

$$= C_{min} + \frac{b}{(2^L - 1)} (C_{max} - C_{min})$$

b = # in decimal form, lets say = 8 and

L = no. of bits, lets say = 6 $C_{min} = -2$, $C_{max} = +5$

for this example $C_i = 0$

∴ within C_{min} and C_{max}, we will get 18 and transform it to 0.

No. of bits, or L is basically discretizing $C_{min} \to C_{max}$.

Ex:-

Data Set :-

x	y
1	1
2	2
3	3
6	6

we want to fit straight line $y = C_1 x + C_2$ to this data

We will create a 12-bit string where first 6 bits C_1 and next 6 $\to C_2$

So starting with 4 members of population randomly generated

1) 000111 010100
2) 010010 001100
3) 010101 101010
4) 100100 001001

C_1	C_2
$7 \to -1.222$	$20 \to 0.222$
$18 \to 0$	$12 \to -0.67$
$21 \to 0.33$	$42 \to 2.67$
$38 \to 2$	$9 \to -1.00$

$-1.222 \to$ transformation $\to 0.222$

We have to find 7 and 20 is equivalent to what in $(-2, +5)$ range. C_{min} and C_{max} have to be defined.

The randomly generated 12-bit string is nothing but selecting C_1 and C_2.

We have to do for all the 4 strings and after C_1 and C_2 are transformed we need see which one are performing the best and reject the weakest.

Sol 1) $G_0 = -1.22$, $G_2 = 0.22$

$y = -1.22x + 0.22$ ... (i)

↳ do it for 2), 3), 4) as well.

We will then use data to calculate y_c from eq(i)

and then $\sum_{\text{4 data}} (y_c - y_t)^2$ ↳ actual value

↳ calculate this for each member of population

Let us also, we will also convert minimisation to maximation problem, by subtracting with very large no.

$\left[400 - \sum_{\text{4 data}} (y_c - y_t)^2\right]$ → for 1st member we get this value as 147.

1) 147
2) 332
3) 391
4) 359

→ unfit most, so discard the 1st member

→ maximum value from 4 members is 3rd member → 391.

To replenish population, replicate 391, so and the copy will replace 1

1) 0101011010 10
2) 0100100011 01
3) 0101011010 10
4) 1001000010 01

→ Here the average maximum value just increased from the previous population.

15/03/24

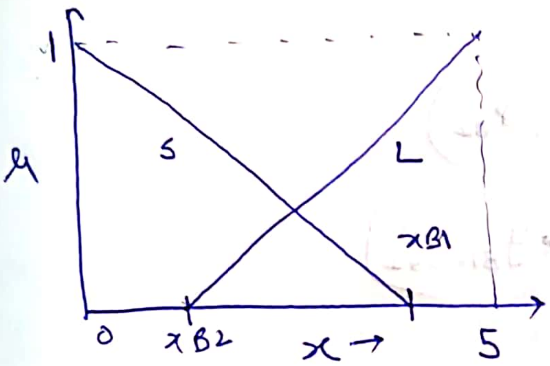
Nothing new

19/03/24 (GA assisted MF for fuzzy system)

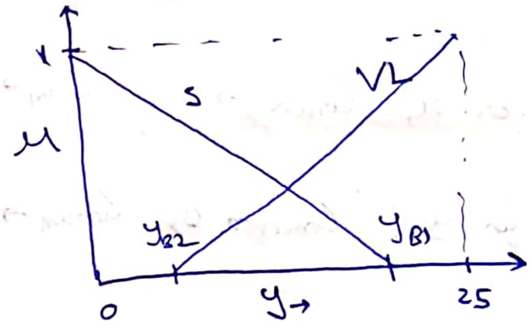
x	1	2	3	4	5
y	1	4	9	16	25

Rule (fuzzy rulebook)

If x	y
S	S
L	VL



(membership function for x)

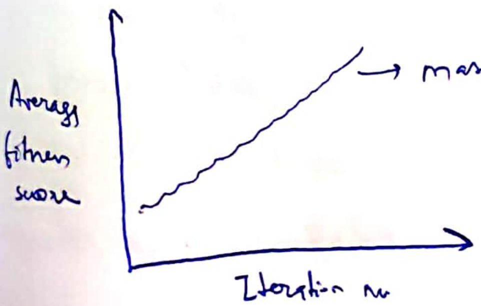


We will use genetic algorithm to optimise the values of x_{B1} and x_{B2}, y_{B1} and y_{B2}, so that both the membership functions are optimized.

GA algorithm steps :- Initialize → Population → Fitness test → Those whose fail discarded → Reproduction of top performing members → Crossover (10-20%) → Mutation (1-5%) → start again → Population

A → A1 A2 — A1 B2

B → B1 B2 ↗ B1 A2



→ maximisation problem, ideally this should be the profile.

	x_{B1}	x_{B2}	y_{B1}	y_{B2}
	6 bits	6 bits	6 bits	6 bits
Decimal equivalent	7	20	22	51
Binary equiv	000111	010100	010110	

So, 24 bit string, 6 bits for each variable. So we similarly generate more population

decimal equivalent

$$\text{Now, } G = G_{min} + \frac{(b)}{(2^L - 1)} (G_{max} - G_{min})$$

So, for x, $G_{min} \to 0$, $G_{max} \to 5$ (x_{B1}, x_{B2})

for y, $G_{min} \to 0$, $G_{max} \to 25$ (y_{B1}, y_{B2})

26/03/24

T.A

8-10 pages report

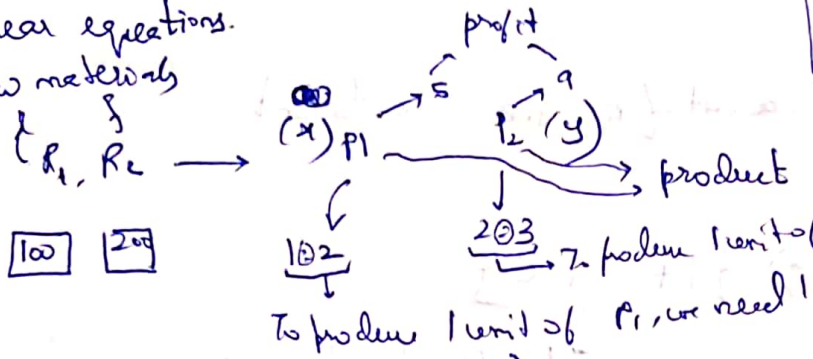
AI - FL/ANN/GA application in chemical eng.

how they are implementing these techniques in industry and see how the systems are working.

Submit by 17/4

Constrained Optimization :-

(1) Linear Programming (LP), where objective function and constraints can be expressed in terms of linear equations.



Objective function :- $(5x + 9y)$ maximize under raw material constraint

So, to produce x unit of P_1, we need x unit of R_1 and $2x$ unit of R_2. To produce y unit of P_2, we need $2y$ units of R_1 and $3y$ units of R_2.

So, $x + 2y \leq 100 \Rightarrow x + 2y \leq 100$

$2x + 3y \leq 200 \Rightarrow 2x + 3y \leq 200$

] → 2 raw material constraints

also we need to consider the non-negativity constraints as well,

$\therefore x \geq 0, y \geq 0$

Similarly, we can have machine/equipment constraints and also manpower constraints.

So, let's say we have a 4 reactor, where P_1, P_2 are produce in both from. Now, per unit of $P_1 \rightarrow$ 3 hrs reactor time

per unit of $P_2 \rightarrow$ 4 hrs reactor time, and total reactor time available is 350 hrs / reactor availability $\therefore 3x + 4y \leq 350$.

a) Minimize $f(x)$

Subj to $g_i(x) \le 0$

$i = 1, 2, \ldots, m$

$g_i(x) \le 0$

c → Violation coefficient

↑ violated

$c_i = g(x)$ if $g(x) > 0$

$= 0$ if $g(x) \le 0$

c → violation coefficient → constraint violated

$c_i = g_i(x)$ if $g_i(x) > 0$

$= 0$ if $g_i(x) \le 0$

$\Rightarrow C = \sum_{i=1}^{m} c_i$

This implies we are satisfying the constraint

$\therefore \phi(x) = f(x)\{1 + K \cdot C\}$

↳ Penalty function ≥ 1

$\phi(x) = f(x)\{1 + K \cdot C\}$

So, the entire constrained problem can be written as,

minimise $\phi(x) = f(x)\{1 + K \cdot C\}$

↳ minimise this

$\therefore$ Q → $x_1 \le x_2$ → also a constraint

minimize :- $(x_1 - 1.5)^2 + (x_2 - 4)^2$

subject to :- $4.5x_1 + x_2^2 - 12 \le 0$ $\Rightarrow 4.5x_1 + x_2^2 - 12 \le 0$

$2x_1 - x_2 - 1 \ge 0$ $\Rightarrow x_2 + 1 - 2x_1 \le 0$

$\Rightarrow -x_1 \le 0$

$\Rightarrow 4 - x_2 \le 0$

$\Rightarrow x_1 - x_2 \le 0$

$x_1 \ge 0, x_2 \ge 4$

→ Convert all of them into form $g(x) \le 0$

convert all constraints of form $g(x) \le 0$

So, to apply GA, we need to randomly select x_1, x_2 and determine the length of each string for both the variables.

(2)

6	6
x_1	x_2

- Now, we need to determine, C_{min} and C_{max} for each variable. They need not be same for all variables.

C_{min}, C_{max} → need not be same for all variables

l → determines the least count / accuracy.

1/04/24

Minimize :-

$$(x_1 - 1.5)^2 + (x_2 - 4)^2$$

Subject to :-

$$4.5x_1 + x_2^2 - 18 \leq 0$$

$$2x_1 - x_2 - 1 \geq 0$$

$$0 \leq x_1, x_2 \geq 4$$

Crossover probability = 30%.

Mutation = 10%.

$g_1(x) \rightarrow 4.5x_1 + x_2^2 - 18 \leq 0$

$g_2(x) \quad x_2 + 1 - 2x_1 \leq 0$

$g_3(x) \quad -x_1 \leq 0$

$g_4(x) \quad 4 - x_2 \leq 0$

→ To formulate $\phi(x)$

So we calculate C_i and then $C = \sum_{\forall i} C_i$

and then $\phi(x) = f(x)\{1 + k \cdot C\}$, $k = 10$

$(x_{1min}, x_{1max}) \to (0, 5)$ $\quad x_1 = \frac{5 \cdot b}{(2^L - 1)} = \frac{5b}{63}$

$(x_{2min}, x_{2max}) \to (4, 10)$ $\quad x_2 = 4 + \frac{6 \times b}{(2^L - 1)} = \left(4 + \frac{6b}{63}\right)$

$= 4 + \frac{6b}{63}$

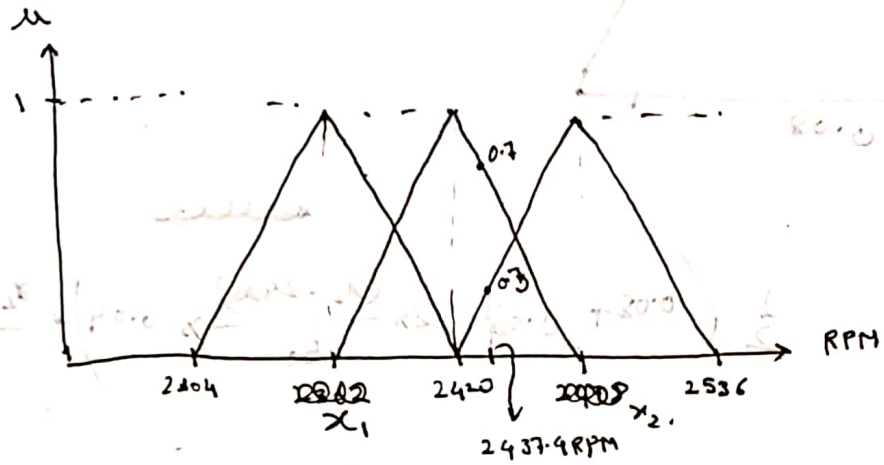
String	x_1 (bin)	x_1	x_2 (bin)	x_2	
000111 010100	7	0.555	20	5.90	(I)
010010 001100	18	1.429	12	5.143	(II)
010101 101010	21	1.667	42	9	(III)
100100 001001	36	2.857	9	4.857	(IV)

	C_1	C_2	C_3	C_4	C
I)	12[illegible]	5.79	0	0	12.79
II)	14.281	3.285	0	0	
III)	1840.5	[illegible]	1	0	0
IV)	225	0	0	0	

$C_1 \quad C_2 \quad C_3 \quad C_4$

2/04/24

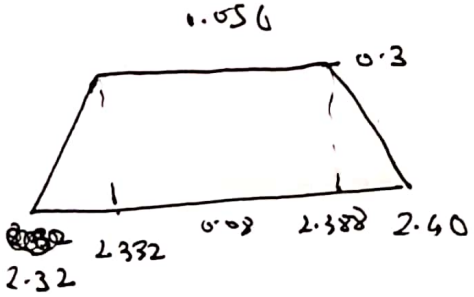
$$\frac{(x_1 - 2420)}{58}$$

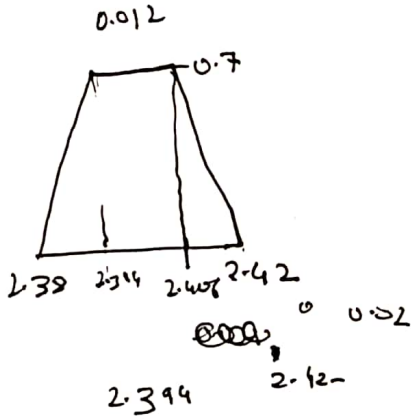


$$\frac{17.4}{58} \quad 20.3$$

0.7 about right → 0.7 just about right

2437.9 RPM → 0.3 [too fast (μ → 0.3)] slow down



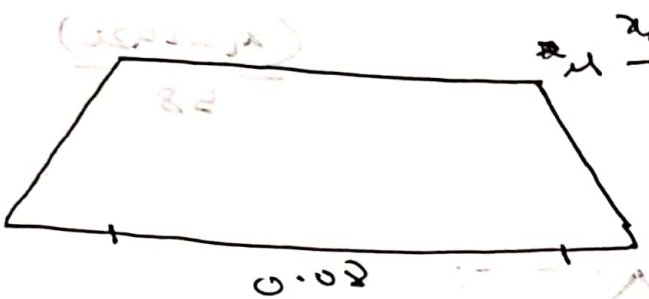


(0.02) 1 → 0.04

$$\frac{0.3}{0.04}$$

$$\frac{0.0204 \times 2.36 + 0.0182 \times 2.4}{0.0254 + 0.0182} \qquad \frac{\frac{1}{2}(0.056 + 0.08) \times 0.3 \times 2.36 + \frac{1}{2}(0.12 + 0.04) \times 0.7 \times 2.4}{2.332}$$

$$= \frac{0.091824}{0.0386} = 2.379$$



$$\frac{1}{2}\left(0.08 + 0.08 - 2\times\frac{(x_2-2400)}{50}\times 0.04\right)\times\frac{x_2-2400}{50}$$

$$4\times 0.04$$

$$(0.08 - 0.08\mu)$$

$$\frac{1}{2}\left[0.08\ (1-\mu) + 0.08\right] 2.4\times\mu \qquad 2\times 2.42$$

Linear Programming

Required 400 l of

rest is coconut oil

Hair oil → 2 comp → Coconut oil
→ Active ingredients. 4% (v/v)

old Stock : → 4.5% AI → atleast 40 l to be used → 0.09 \$/l
→ 3.7% AI → 0.07 \$/l

pure coconut oil can be added free of cost.

The above three things can be added.

How many liters of A, B C and will blend to get 400 l of 4% (v/v) with min cost.

(min)
Optimize :- $0.09 V_1 + 0.07 V_2$

Constraints :- $V_1 + V_2 + V_3 = 400$

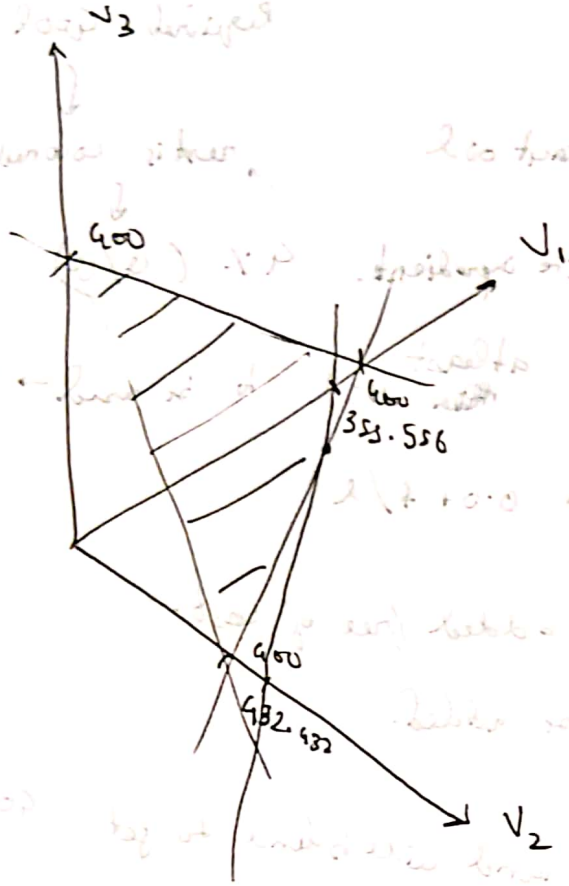
$0.045 V_1 + 0.037 V_2 \geq 400 \times 0.04 \Rightarrow 0.045 V_1 + 0.037 V_2 = 16$

$V_1 \geq 40$

$V_1 + V_2 + V_3 = 400$

$0.045 V_1 + 0.037 V_2 = 16$

$V_1 \geq 40, \quad V_2 \geq 0, \quad V_3 \geq 0$



$V_1 + V_2 = 400$

$(400 - V_2) \times 0.045 + 0.08 V_2 = 16$

$(18 - 16) = (0.08) V_{12}$

250 V_2

150 V_1

Products P1 and P2

Batch Process of two steps

1000 kg capacity

Step 1 → 5000 hrs/yr

Step 2 → 6000 hrs/yr

P1 → 3$/kg

P2 → 2$/kg

	Reactor S1-1	Reactor S1-2
P1	25 hrs	10 hrs
P2	10 hrs	20 hrs

if each of these is to be produced in 1 batch.
↳ 1000 kg.

No. of batches - $P_1 \rightarrow x$

No. of batches Per y

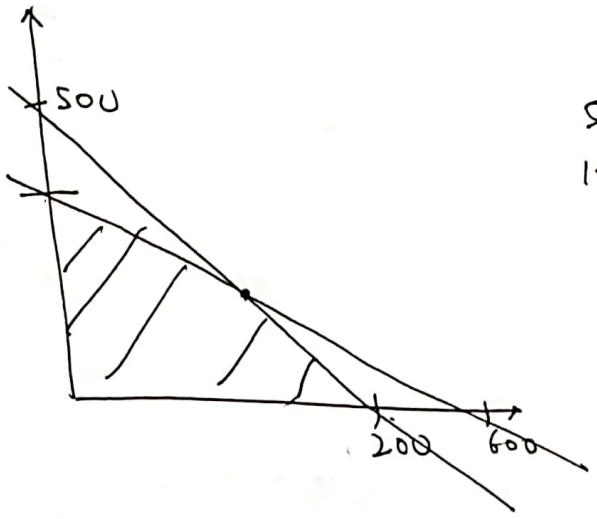
$1000 \times x \times 3$

Maximise :- $3000x + 2000y$

Constraints :- $(25x + 10y) \leq 5000$

$(10x + 20y) \leq 6000$

$x \geq 0, y \geq 0$



$50x + 20y = 10000$

$10x + 20y = 6000$

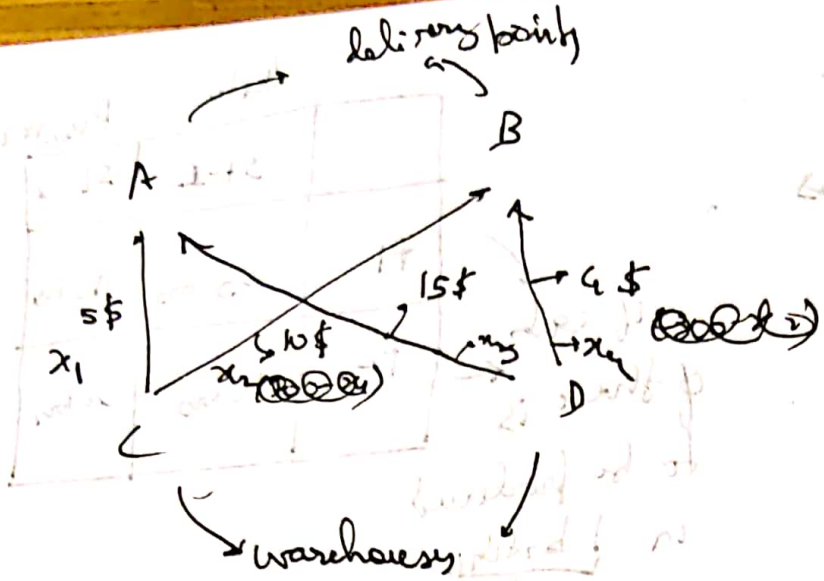
$40x = 4000$

$x = 100$

$y = 250$

800,000$

Q3)



$5x_1 + 10x_2 + 15x_3 + 4x_4$

$5x_1 + 10x_2 + 15x_3 + 4x_4$

Maximize :- $10200 + 5x_1 + 11x_2$

Maximize :- $5x_1 + 10x_2 + 15x_3 + 4x_4$

Constraint → $x_1 + x_2 = 800$

$1500 - x_1 - x_2 =$

Constraint $x_1 + x_3 = 600$

$x_2 + x_4 = 400$

$x_1 + x_2 \leq 700$

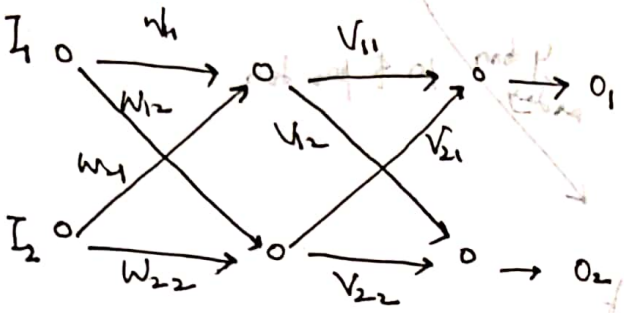
$x_3 + x_4 \leq 800$

8/04/24 (example of non-binary string)

GA → to optimise the weights of the ANN

m n l
2 × 2 × 2

$(m \times n) + (n \times l)$ → no. of weights



$E = (T_1 - O_1)^2 + (T_2 - O_2)^2$

Preprocessing
→ plausibility check
→ normalisation
→ Randomisation.
→ Time delay analysis

I_1	I_2	T_1	T_2
–	–	–	–
–	–	–	–
–	–	–	–

2/3 Training

1/3 Testing

W_{11} W_{12} W_{21} W_{22} V_{11} V_{12} V_{21} V_{22} → Total 40 bit decimal string.

Sign bit

X X X X X → 5 bit string with each X allowed to be from 0-9.

decimal string.

* If first bit between 0 → 4, then –ve
else, +ve.

84321 / 46234 / 78901 / . . .

84321 corresponds to +4.321

This weight is +ve.

This weight is +ve

This weight is negative

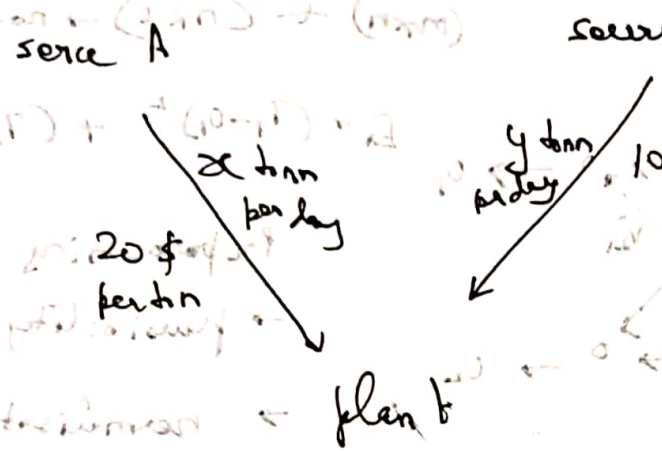
46234 → −6.234

78901 → +8.901

Each string will have a fitness $f_i = \frac{1}{E_i}$

(40-)

a)



min → 3 tons.

Cost :- $20x + 10y \leq 80$

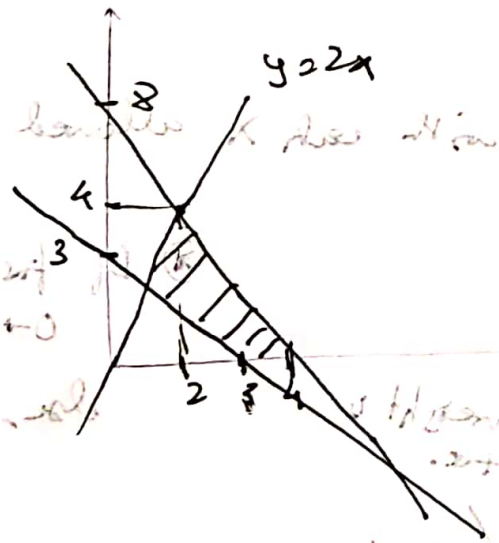
$x + y \geqslant 3$

$y \leq 2x$

$(2x + 3y)$ oz of gold from both the sources

objective function

maximize



$x = 2$.

$2 + 6 \quad 8$